

Módulo 8

M8.1 Hierarquias de classes

Para interligar as classes usa-se estruturas como: generalização e especialização, fazendo com que os atributos e serviços comuns fiquem claros em uma hierarquia de Classes.

Então quando se dispõe de mecanismos de herança - disponíveis apenas em linguagens orientadas a objeto, a especificação de uma classe pode ser retirada de uma biblioteca existente e uma nova classe (Subclasse) pode ser criada a partir de uma classe pai (superclasse), através do processo de derivação. A classe filha pode especificar as suas características que diferem da classe pai.

Um problema apresentado pela herança é que uma subclasse pode herdar métodos (na maioria das vezes) que não necessita ou que não deveria os possuir, então o arquiteto de software deve assegurar que os recursos fornecidos por uma classe são apropriados para futuras subclasses.

Contudo, então, a melhor idéia seria que um programador possuísse um conjunto de bibliotecas de classes com código já implementado, à sua disposição, e então definiria quais destas seriam úteis para a confecção do novo software, tornando a implementação mais simples. Este procedimento é chamado de reuso.

Vamos imaginar o seguinte exemplo :

- Estudante é uma Pessoa
- Professor é uma Pessoa.
- Um objeto da classe Estudante, tem os atributos e serviços de Estudante, e também os atributos herdados da classe Pessoa.

Por exemplo, João é um estudante e tem os atributos e serviços de Pessoa, como: Nome, Telefone, Andar, Viajar.

Além destes possui atributos e serviços que expressam o comportamento específico de estudante, como: Curso, Nota, Estudar e Realizar Prova.

A classe Pessoa é reconhecida como uma generalização e Estudante como uma classe especialização a qual herda os serviços e atributos de Pessoa. Além disso, uma classe derivada (uma especialização) pode incluir novos serviços e atributos ou redefinir os serviços herdados.

Desta mesma forma poderíamos ter também, a classe Professor, Diretor, Funcionário, Reitor, etc, todas derivadas da classe Pessoa, tendo suas características específicas declaradas em sua própria estrutura, e fazendo uso das características e operações genéricas declaradas na superclasse (classe base).

Com isso, concluímos que a Hierarquia de Classe é composta da Generalização, representada pela Classe Bases e pela Especialização, representada pela Classes Filha.

Quando a modelagem de um sistema em POO chega à necessidade de derivação entre classes (herança) é altamente aconselhável que a classe pai (classe principal) seja constituída da maior parte possível dos métodos e atributos que serão utilizadas das derivações (classes filhas). Este cuidado previne redundância de informações e com isso facilita a manutenção do sistema.

Logo, geração de um conjunto de derivações partidas de uma classe pai tem o nome de hierarquia de classes.

Um projeto de um sistema OO começa sempre com a definição da hierarquia de classes descrevendo as relações dos objetos que irão formar o programa.

É comum, quando estudamos uma linguagem de programação orientada a objetos, estudarmos a hierarquia de suas classes. Este estudo ajuda a entender o funcionamento da linguagem de programação, conhecendo assim as classes já existentes e evitando retrabalho.

Classes que declaram mas não implementam métodos são particularmente úteis na criação de hierarquias de classes, porque não permitem a criação de instâncias delas e exigem que as classes descendentes implementem os métodos declarados nelas.

Neste momento é interessante estudar a classe **Object**. Ela suporta todas as classes na hierarquia de classes e fornece serviços de baixo nível para classes derivadas. A classe **Object** é classe base fundamental de todas as classes; ela é a raiz da hierarquia de tipos.

Como todas as classes são derivadas da classe **Object**, todo método definido na classe **Object** está disponível em todos os demais objetos no sistema. A classe **Object** é a superclasse de todas as classes, e todas as classes são herdeiras dela..

Na Tabela abaixo temos os métodos da classe **Object**, eles serão herdados nas classes derivadas que podem sobrescrever e sobrescrevem alguns desses métodos já que elas são declaradas virtuais

Nome	Descrição
Equals(Object)	Verifica se o objeto é igual ao objeto atual
Equals(Object, Object)	Determina se as instâncias dos objetos são iguais.
Finalize	Permite um objeto tentar liberar recursos e executar outras operações de limpeza antes que ele seja recuperado pelo Garbage Collector
GetHashCode	Serve como a função de hash padrão.
GetType	Obtém o Tipo da instância atual.
MemberwiseClone	Cria uma cópia do Object atual
ReferenceEquals	Determina se as instâncias especificadas de Object é a mesma instância.
ToString	Retorna uma cadeia que representa o objeto atual

Vamos construir então uma classe (Ponto) utilizando alguns métodos da classe *Object*. No método sobrescrito *Equals()* ele recebe como parâmetro um objeto do tipo *object*, dentro dele é utilizado o método *GetType()* para obter o tipo da instância corrente. No método sobrescrito *GetHashCode()* é feito uma simulação do cálculo de um *Hash* particular para caracterizar o objeto e não o valor devolvido pelo método padrão. O último método sobrescrito o *ToString* já foi utilizado em alguns programas exemplos mas sem explicação, e esclarecendo agora. Uma *string* pode ser mostrada quando simplesmente solicitamos para escrever na saída quando simplesmente colocamos o nome da instância. Por padrão o método *ToString* mostra o caminho da herança do objeto. A classe ainda tem um método *Copy* que utiliza o método *memberWiseClone()* que cria uma nova instância do objeto corrente.

```
class Ponto
{
    public int x, y;

    public Ponto(int x, int y)
    {
        this.x = x;
        this.y = y;
    }

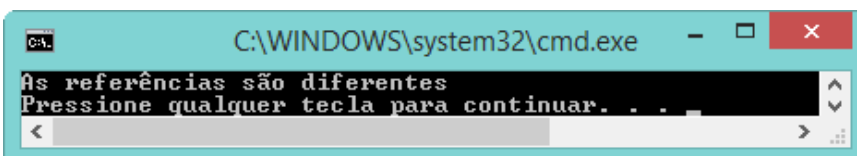
    public override bool Equals(object obj)
    {
        if (obj.GetType() != this.GetType())
            return false;
        Ponto other = (Ponto)obj;
        return (this.x == other.x) && (this.y == other.y);
    }

    public override int GetHashCode()
    {
        return x ^ y;
    }
    public override String ToString()
    {
        return String.Format("{0}, {1}", x, y);
    }

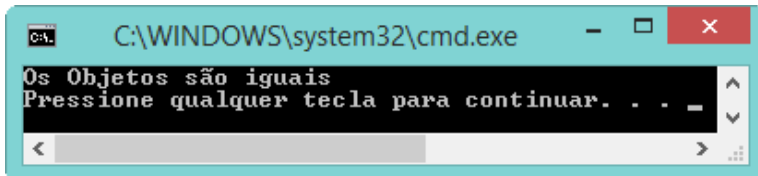
    public Ponto Copy()
    {
        return (Ponto)this.MemberwiseClone();
    }
}
```

Testando os diversos métodos:

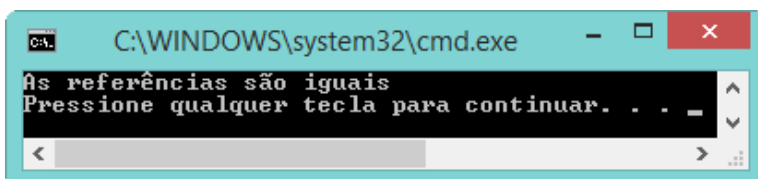
```
class Program
{
    static void Main(string[] args)
    {
        Ponto p1 = new Ponto(1, 2);
        Ponto p2 = p1.Copy();
        if (Object.ReferenceEquals(p1, p2))
            Console.WriteLine("As referências são iguais");
        else
            Console.WriteLine("As referências são diferentes");
    }
}
```



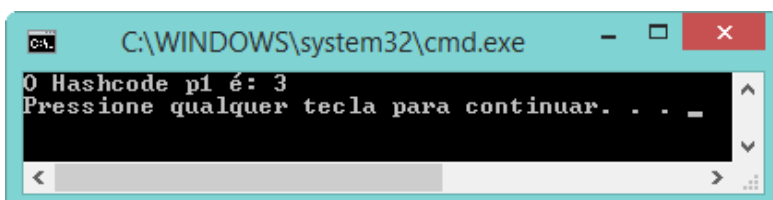
```
class Program
{
    static void Main(string[] args)
    {
        Ponto p1 = new Ponto(1, 2);
        Ponto p2 = p1.Copy();
        if (Object.Equals(p1, p2))
            Console.WriteLine("Os Objetos são iguais");
        else
            Console.WriteLine("Os Objetos são diferentes");
    }
}
```



```
class Program
{
    static void Main(string[] args)
    {
        Ponto p1 = new Ponto(1, 2);
        Ponto p3 = p1;
        if (Object.ReferenceEquals(p1, p3))
            Console.WriteLine("As referências são iguais");
        else
            Console.WriteLine("As referências são diferentes");
    }
}
```



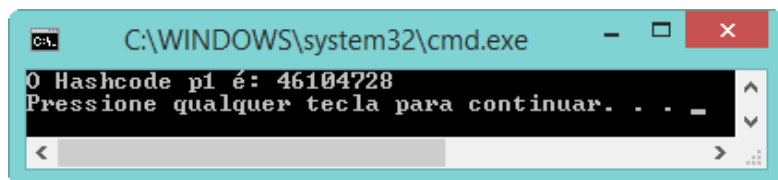
```
class Program
{
    static void Main(string[] args)
    {
        Ponto p1 = new Ponto(1, 2);
        Console.WriteLine("O Hashcode p1 é: {0}", p1.GetHashCode());
    }
}
```



```

/*      public override int GetHashCode()
{
    return x ^ y;
}
*/

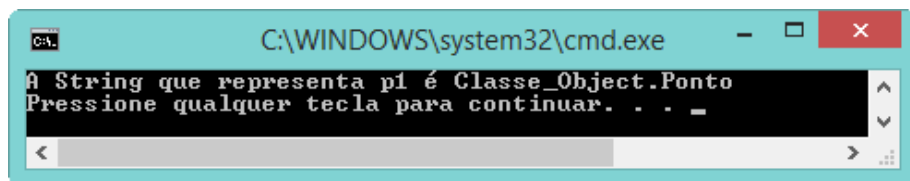
```



```

/*      public override String ToString()
{
    return String.Format("{0}, {1})", x, y);
}
*/

```



Desta forma ao projetarmos um sistema OO para ter sempre a hierarquia íntegra devemos sempre termos as seguintes considerações:

- • Todo objeto é instância de uma e só uma classe, um objeto não pode ser instanciado a partir de duas classes.
- • Todos os objetos de uma mesma classe:
- • Cada um tem um conjunto de atributos de informação, que determinam o "estado" do objeto a cada momento, que são as variáveis de instância ou campos, mas independentes entre si desde que não estáticos..
- • Cada um tem um conjunto de funcionalidades - "ações" ou "serviços" que podem ser solicitados por meio de "mensagens", e executadas por métodos de mesmo
- • Duas classes diferentes precisam ser distintas: ter algum atributo, ou algum método, ou ambos diferentes. Caso contrário, não teria sentido nomeá-las de forma diferentemente se são iguais
- • Duas classes que são subclasses de uma superclasse herdam os atributos e métodos comuns da superclasse, mas possuem outros atributos e métodos que as particularizam.
- • Em muitas linguagens orientadas a objetos, as classes formam uma Hierarquia Simples. Ou seja, há uma raiz de todas as classes, que é a classe Object. Todas as demais classes têm necessariamente uma única superclasse, mas terem ou não várias subclasses. Essa estrutura é a de uma árvore invertida.
- • Existe uma relação "é-um" entre um objeto e sua classe, que "é um" para a superclasse desta, e propaga para todas as superclasses acima na hierarquia até a

classe *Object*. (por exemplo: um tigre "é um" mamífero, que "é um" animal que "é um" objeto)

- Todo objeto é instância de uma classe. Um objeto obedece, através de uma referência o comportamento da classe.
- Podemos redefinir, em uma subclasse, um método com a mesma assinatura de outro já existente na sua superclasse, ou mais acima na hierarquia. Nesse caso, esse novo método passará a substituir o método da superclasse. Esse mecanismo chama-se "overriding", ou sobrescrita, e é uma das formas de polimorfismo.
- Um atributo de uma classe pode referenciar qualquer instância dessa classe, e qualquer instância das suas subclasses. Mas não pode referenciar objetos de sua superclasse, ou de outras classes acima na hierarquia.
- Quando um objeto recebe uma mensagem, inicialmente procura um método com a mesma assinatura. Se encontrar, executa esse método. Caso não encontre, passa a procurar na sua superclasse, na superclasse dela, e assim por diante, executando o primeiro método que encontrar com a mesma assinatura. Caso alcance a classe *Object*, e nem lá encontre esse método, ocorrerá um erro de execução

M8.2 Inicialização e destruição de objetos

É importante saber que a compreensão sobre o tempo de vida de um objeto está ligada a como a linguagem de programação lida com esses objetos em memória.

Linguagens de POO mais antigas costumam acumular os objetos criados, deixando a cargo do programador a realizar a destruição do mesmo quando não mais utilizar. Neste caso, vale salientar que não adianta finalizar a execução do programa principal, pois os objetos instanciados dentro do programa e que não foram destruídos no momento oportuno, continuam na memória. Assim sendo, a única maneira de limpar a memória desses objetos é desligando o computador.

Linguagens de POO mais recentes, possuem ferramentas que possibilitam que, uma vez o objeto instanciado fique sem utilização (sem referências), tais ferramentas se encarregam de limpar a memória desses resíduos. Não por acaso, essas ferramentas possuem nomes sugestivos como "Coletores de lixo".

Embora essas linguagens realizam esse trabalho de destruição de objetos que não estejam mais sendo utilizados, é muito salutar que o desenvolvedor crie o hábito de se preocupar, pelo menos no momento do desenvolvimento da classe, como será a destruição da classe.

Assim como toda a classe possui um método especial chamado construtor, cujo objetivo é definir processos que ocorrerão unicamente quando o objeto for instanciado na memória, existe também um método especial chamado destruidor (ou desconstrutor). Esses métodos destruidores tem o objetivo de definir processos que serão executados apenas quando o objeto for removido da memória, ou seja, destruído.

Esses métodos tornam-se muito importantes a medida que a complexidade do seu projeto aumenta, pois podemos utilizar os processos desses métodos para liberação ou alocação de recursos em dispositivos (impressora, porta serial, etc...), controle de arquivos, conexão e desconexão ao banco de dados, etc...

Portanto, o tempo de vida de um objeto abrange desde sua instancia em memória até sua liberação, passando pelos processos de criação e destruição, que devem ser definidos na elaboração da classe.

Conforme vimos anteriormente, um objeto é uma instância de uma classe. Utilizando o Visual Basic .Net, utilizamos a palavra-chave *New* para esta tarefa.

Vejamos um exemplo :

Dim mObjeto As New MeuObjeto

Já em C#, a mesma instrução seria :

MeuObjeto mObjeto = new MeuObjeto();

Apesar da diferença de sintaxe, o conceito é exatamente o mesmo : um classe MeuObjeto é instanciada e sua referência é a variável mObjeto. Analisando este cenário, percebemos que uma determinada classe faz uso de outra classe (MeuObjeto). Quando o objeto não é mais utilizado pela aplicação, o mesmo pode ser manualmente apagado. Em Visual Basic, utilizamos a instrução Nothing para esta finalidade, conforme apresentado abaixo.

mObjeto = nothing

Já em C#, teríamos :

mObjeto = null;

Quando um objeto não é mais utilizado, dizemos que ele não faz mais parte do contexto. Com o intuito de liberar recursos do sistema, o Common Language Runtime (CLR) - mecanismo responsável pela execução das aplicações .NET Framework, se encarrega de verificar quais objetos estão fora de contexto e, através do Garbage Collection (GC), os remove da memória.

Garbage Collector (GC)

O garbage collector, ou coletor de lixo, muitas vezes é ignorado pelos programadores e analistas, mas é um ponto importante a ser considerado.

O princípio básico de como um garbage collector funciona é:

1. Determinar que objetos não mais serão utilizados no futuro.
2. Liberar os recursos utilizados por esses objetos

Em linguagens mais antigas, esses recursos deveriam ser liberados manualmente, ficando a cargo do programador o controle de objetos não utilizados.

Com o garbage collector, encontrado em linguagens atuais como, por exemplo, JAVA, C# e a grande maioria das linguagens script, não temos mais que nos preocupar com a liberação de recursos enquanto desenvolvemos (com exceção para celulares, ou aparelhos com hardware limitado , porém, será que a liberação manual não nos trás alguns benefícios? Com certeza !

Deixar o garbage collector trabalhar por nós significa perder processamento para verificar quais objetos não serão mais utilizados no futuro, o que não é uma tarefa tão simples se pensarmos em lógica.

Quando liberamos recursos no sistema manualmente, esse processo deixa de ser longo, logo, o processamento utilizado pelo garbage collector é menor e sua aplicação vai rodar mais rápido.

Em compensação, o programador tem um trabalho maior e a chance de bugs acontecerem também aumenta.

- * Bugs causados por ponteiros
- * Double free - Bug causado pela tentativa de liberar memória quando essa já está livre.
- * Memory leaks - Quando a tentativa de liberar memória falha, causando sua exaustão.

O gerenciamento automático de memória, conhecidos como garbage collector ou simplesmente coletores, é um serviço, que automaticamente, libera blocos de memória que não serão mais usados em uma aplicação.

As vantagens desse tipo de gerenciamento são:

- * liberdade do programador que não está mais obrigado a ficar atento a detalhes de memória;
- * há menos bugs de gerenciamento de memória;
- * gerenciamento automático é mais eficiente que o manual.

Entre as desvantagens, podemos citar:

- * desenvolvedor tende a estar mais desatento em relação a detalhes de memória;
- * gerenciador automático apresenta limitações.

Um objeto está apto a ser usado pelo GC quando não há mais referências a esse objeto. O ambiente .NET e Java tem o GC que periodicamente libera memória usada por objetos que estão há algum tempo sem serem referenciados. Essa atividade é automática, entretanto, em algumas situações, o desenvolvedor pode querer executar o coletor explicitamente, chamando o método GC. Por exemplo, você poder querer executar o GC após um setor de código que cria um grande número de objetos que não serão usados mais ou antes de um trecho de sua aplicação que necessitará muita memória disponível.

M8.3 Tratamento de Exceções

Em qualquer sistema, OO ou não, não basta que ele apenas esteja com a programação bem feita. Ele também deverá estar preparado para tratar de problemas (exceções) que venham a ocorrer. Nenhum sistema está imune às exceções, e em algum momento qualquer sistema deverá estar preparado para tratar de um problema. Uma exceção em tempo de execução poderia encerrar o sistema de forma abrupta mostrando uma mensagem incompreensível levando o usuário ficar sem ação, ou mesmo danificando os dados que estavam sendo processados.

Para tratar estes problemas é importante conhecer algumas técnicas de controle de erros, chamada de controle de exceção.

A linguagem C# adota um estilo relativamente comum em outras linguagens, que é o de tratar erros como objetos que encapsulam todas as informações que precisamos para resolvê-lo. Essa ideia é válida para todo o ambiente .NET e foi batizada como SEH (Structured Exception Handling) (Lima e Reis, 2002). O formato básico para tratamento de uma exceção é o seguinte.


```

try
{
    // Código sujeito a exceções
}
catch (TipoExcecao1 e)
{
    // Tratamento para exceção tipo 1
}
catch (TipoExcecao2 e)
{
    // Tratamento para exceção tipo 2
}
catch
{
    // Tratamento para qualquer tipo de exceção
}
finally
{
    // Trecho que deve sempre ser executado, havendo ou não exceção
}

```

A estrutura *try..Catch* requer que exista pelo menos um bloco *Catch* vazio. Os blocos de *Finally* e *Catch(exception)* são opcionais. Os comandos de tratamento de exceções está na tabela abaixo.

Comando	Descrição
<i>try</i>	Intervalo do código onde pode ocorrer o erro
<i>Catch</i>	Intervalo de código onde o erro será tratado
<i>Finally</i>	Essa instrução é executada sempre, utiliza-se para liberar algum recurso independente da ocorrência do erro
<i>Throw</i>	Comando que gera exceções específicas

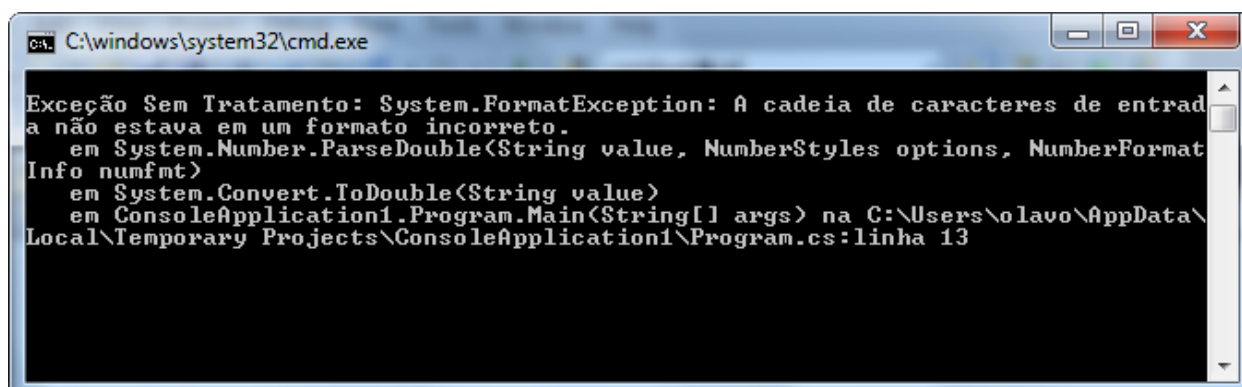
Temos um programa que ao digitar no editor não acusa nenhum erro..

```

class Program
{
    static void Main(string[] args)
    {
        String v = "ABC";
        double num = Convert.ToDouble(v);
        Console.WriteLine(num);
    }
}

```

porém ao executar o programa acontece um erro:



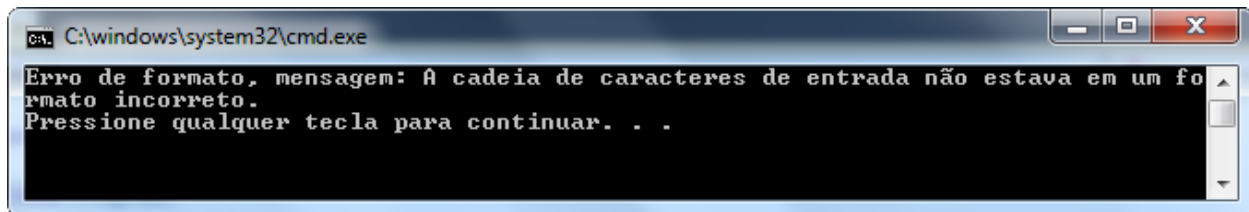
O erro acontece porque a função `Convert.ToDouble()`, que transforma uma *string* em um número do tipo *Double*, está tentando transformar uma *string* que não contém um número ("ABC"). Esse tipo de erro ocorre muito quando é feita a leitura do teclado de um número, e a função `Console.ReadLine()` retorna uma *string* que necessita ser convertido. O usuário poderia digitar uma palavra ao invés de digitar um número. A maneira de fazer corrigir é fazer um tratamento deste erro.

O trecho problemático deve ser envolvido pelo bloco do *try*. No bloco do *Catch* é feito o tratamento do erro (Código 100). Note que logo após a palavra *Catch* é feita a classificação do tipo de erro, desta forma o tratamento do erro é específico para um tipo de erro (no caso *FormatException*). Um *try* pode ter vários *Catch* cada um deles tratando um tipo de erro. O C# contém uma série de classes para manipulação de exceções e é uma boa prática utilizamos essas classes

Exception	Descrição
<code>DivideByZeroException</code>	Erro gerado quando dividimos números inteiros por zero.
<code>IndexOutOfRangeException</code>	Erro gerado quando acessamos posições inexistentes de um <i>array</i>
<code>FormatException</code>	Erro gerado quando acontece um problema na conversão
<code>NullReferenceException</code>	Erro gerado quando utilizamos referências nulas
<code>InvalidCastException</code>	Erro gerado quando realizamos um <i>casting</i> incompatível

Tratando um erro de formato numérico

```
class Program
{
    static void Main(string[] args)
    {
        String v = "ABC";
        try
        {
            double num = Convert.ToDouble(v);
            Console.WriteLine(num);
        }
        catch (System.FormatException err)
        {
            Console.WriteLine("Erro de formato, mensagem: " + err.Message);
        }
    }
}
```



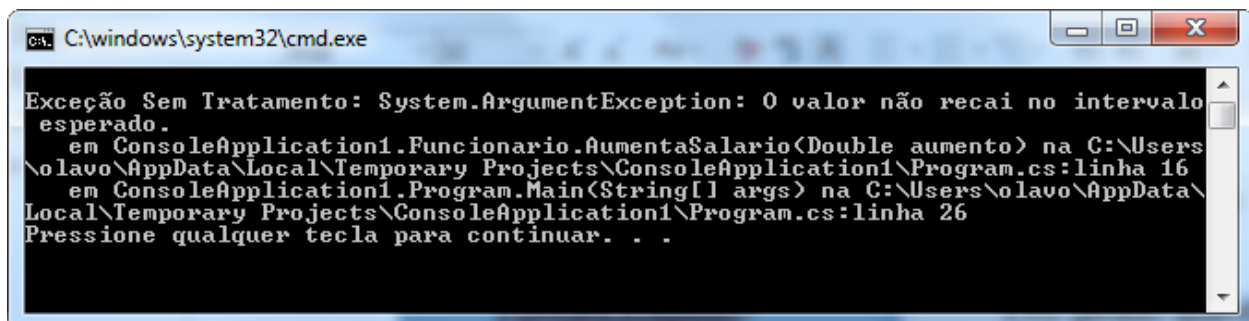
Existem ocasiões que podemos utilizar o tratamento de erros como recurso de programação, e não somente para erros de lógica. Nestas ocasiões podemos fazer o programa gerar propositalmente o próprio erro. Quando precisamos gerar as nossas próprias exceções utilizamos a instrução *Throw*, geralmente a instrução *Throw* é utilizada quando queremos fazer alguma validação.

No exemplo vamos provocar um erro da classe *ArgumentException* quando em um método que calcula o aumento de salário é passado um número negativo. Normalmente a ocorrência de um erro neste tipo de rotina é raro, mas é interessante provocar um erro quando alguma não queremos que uma situação particular ocorra.

```
class Funcionario
{
    public void AumentaSalario(double aumento)
    {
        if (aumento < 0)
        {
            ArgumentException erro = new ArgumentException();
            throw erro;
        }
    }
}
```

Sem tratamento de erro:

```
class Program
{
    static void Main(string[] args)
    {
        Funcionario f = new Funcionario();
        f.AumentaSalario(-1000);
    }
}
```

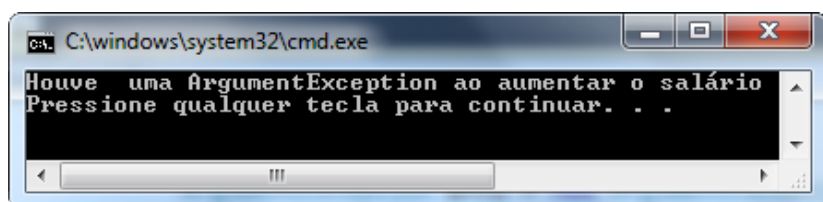


Tratando o erro:

```

class Program
{
    static void Main(string[] args)
    {
        Funcionario f = new Funcionario();
        try
        {
            f.AumentaSalario(-1000);
        }
        catch (ArgumentException e)
        {
            System.Console.WriteLine("Houve uma ArgumentException ao aumentar o salário");
        }
    }
}

```



Exemplo de um programa que testa entrada e divisão por zero:

```

public class Circulo
{
    private readonly float PI = 3.141592654f;
    private float raio;
    private string erro = "";
    public virtual float solicitaRaioDoCirculo()
    {
        try
        {
            Console.WriteLine("Digite o valor do raio do círculo: ");
            raio = Convert.ToSingle(Console.ReadLine());
        }
        catch (FormatException e)
        {
            Console.WriteLine("Comente sempre raio não válido:");
            erro = e.Message;
            Console.WriteLine("Erro: " + erro);
        }
        finally
        {
            if (erro != "")
            {
                raio = 0.0f;
            }
        }
        return raio;
    }
    public virtual void calculaPerimetroDoCirculo()
    {
        float perimetro;
        perimetro = 2 * PI * raio;
        Console.WriteLine("Perimetro: " + Convert.ToString(perimetro));
    }
    public virtual void calculaAreaDoCirculo()
    {
        float area;
        area = PI * raio * raio;
        Console.WriteLine("Area: " + Convert.ToString(area));
    }
}

42 public class Divisao
43 {
44     private int divisor, dividendo;
45     public virtual bool calculaDivisao()
46     {
47         int divisao;
48         bool resultado;
49         string erro;
50         try
51         {
52             Console.WriteLine("Digite o valor do divisor: ");
53             divisor = Convert.ToInt32(Console.ReadLine());
54             Console.WriteLine("Digite o valor do dividendo: ");
55             dividendo = Convert.ToInt32(Console.ReadLine());
56             divisao = (int)(dividendo / divisor);
57             Console.WriteLine("Divisao: " + Convert.ToString(divisao));
58             resultado = true;
59         }
60         catch (ArithmeticException e)
61         {
62             Console.WriteLine("Divisao por zero não permitida:");
63             erro = e.Message;
64             Console.WriteLine("Erro: " + erro);
65             resultado = false;
66         }
67         catch (FormatException e)
68         {
69             Console.WriteLine("Comente sempre raio não válido:");
70             erro = e.Message;
71             resultado = false;
72         }
73         return resultado;
74     }
75 }
76
77
78
80 class Program
81 {
82     public static void Main(String[] args)
83     {
84         // TODO Auto-generated method stub
85         bool naoAcerta = false;
86         Circulo circulo = new Circulo();
87         circulo.solicitaRaioDoCirculo();
88         circulo.calculaAreaDoCirculo();
89         circulo.calculaPerimetroDoCirculo();
90         Console.WriteLine();
91         Console.WriteLine();
92         while (!naoAcerta)
93         {
94             Divisao divisao = new Divisao();
95             naoAcerta = divisao.calculaDivisao();
96         }
97         Console.WriteLine();
98         Console.WriteLine("FIM DO PROGRAMA");
99     }
100 }
101

```

M8.4 Material complementar

Compreensão dos fundamentos da programação em linguagem C#

Em Junho de 2000, a Microsoft anunciou a plataforma .NET e a nova linguagem de programação chamada C#. C# é uma linguagem orientada a objetos fortemente tipada desenvolvida para criar uma combinação de simplicidade, expressividade e performance. A plataforma .NET é centrada na Common Language Runtime (similar à JVM) e um conjunto de bibliotecas que podem ser aproveitadas em uma grande variedade de linguagens que são capazes de trabalhar em conjunto durante a compilação para uma linguagem intermediária IL (Intermediate Language). C# e .NET são uma espécie de simbiose: algumas características do C# existem para trabalhar bem com o .NET, e algumas características do .NET existem para funcionar bem com o C# (embora o .NET também trabalhe bem com muitas outras linguagens). A linguagem C# foi construída através da observação de muitas outras linguagens, mas mais notavelmente Java e C++. Entre seus criadores estão Anders Hejlsberg (que é famoso pelo projeto da linguagem utilizada no Delphi) e Scott Wiltamuth.

C# e Java

Abaixo está uma lista das características que o C# e o Java compartilha e quais tentaram melhorar do C++.

- * São compiladas para um código independente de linguagem e máquina que é executado em um ambiente de execução controlado.
- * Coleta de lixo e eliminação de apontadores (em C# um uso restrito é permitido dentro do código marcado como não-seguro (unsafe))
- * Poderosas capacidades de reflexão.
- * Sem arquivos de header, todo o código fica em packages ou assemblies, sem problemas de declarar uma classe antes de outra com dependências circulares
- * Todas as classes descendem de object e devem ser alocadas no heap com new.
- * Suporte a threads colocando um bloqueio nos objetos quando eles entram em código marcado como locked/synchronized
- * Interfaces, com herança múltipla de interfaces, herança simples de implementações
- * Classe internas
- * Nenhum conceito de herança de uma classe com um nível de acesso específico.
- * Sem funções ou constantes globais, tudo pertence a uma classe.
- * Arrays e strings com tamanhos embutidos e checagem de limites.
- * O operador "." é sempre usado, não mais os operadores -> e ::
- * null e boolean/bool são palavras-reservadas
- * Todos os valores são inicializados antes do uso
- * Não se pode usar inteiros para decidir instruções if
- * Blocos Try podem ter uma cláusula finally.

Properties

Properties serão um conceito familiar para os usuários do Delphi e do Visual Basic. Eles existem para a linguagem formalizar o conceito de métodos get e set, que são bastante usados, particularmente em ferramentas de RAD (Rapid Application Development).

Este é um código comum que você poderia escrever em Java ou C++:

```
foo.setSize (getSize () + 1);  
label.getFont().setBold (true);
```

O mesmo código seria escrito assim em C#:

```
foo.size++;  
label.font.bold = true;
```

O código em C# é imediatamente mais legível por aqueles que estão usando foo e label. Existe simplicidade semelhante quando se implementa properties.

Java/C++:

```
public int getSize() {  
    return size;  
}  
  
public void setSize (int value) {  
    size = value;  
}
```

C#:

```
public int Size {  
    get {return size;  
    }  
    set {size = value;  
    }  
}
```

Particularmente, para ler/escrever properties, C# fornece uma forma mais clara de manipular este conceito. O relacionamento entre um método get e set é inerente em C#, enquanto que em Java ou C++ ele tem que ser desenvolvido. Há muitos benefícios nessa abordagem. Ela encoraja os programadores a pensar em termos de propriedades, e se aquela propriedade é mais natural como leitura/escrita do que como somente leitura, ou se ela realmente deveria ser uma propriedade. Se você quiser alterar o nome de sua propriedade, você tem que olhar em apenas um lugar (get's e set's às vezes consomem centenas de linhas de código). Os comentários têm que ser feitos apenas uma vez, e não ficarão fora de sincronia um com o outro. É possível que uma IDE possa ajudar aqui, mas devemos nos lembrar de um princípio essencial em programação que é tentar fazer abstrações que modelem bem o escopo do nosso problema. Uma linguagem que suporta properties colherá os benefícios daquela melhor abstração.

Um possível argumento contra o fato disto ser um benefício é que você não sabe se está manipulando um campo ou uma propriedade com esta sintaxe. Contudo, quase todas as classes com algum nível real de complexidade em Java (e certamente em C#) não tem campos públicos. Campos geralmente têm um nível de acesso reduzido (private/protected/default) e são expostos apenas através de get's e set's, o que significa que se poderia ter aqui uma sintaxe melhor. Também é totalmente possível que uma IDE possa analisar o código, iluminando properties com uma cor diferente, ou informação adicional sobre o código indicando se é uma property ou não. Também deve ser notado que se uma classe é bem projetada, então um usuário da classe deveria se preocupar apenas com a especificação daquela classe, e não com sua implementação. Indexadores C# fornece indexadores que permitem que objetos seja tratados como arrays, exceto para propriedades, cada elemento é exposto com um método get e/ou set.

```
public class Buildings  
{  
    Story[] stories;  
    public Story this [int index] {  
        get {  
            return stories [index];  
        }  
        set {
```

```

        if (value != null) {
            stories [index] = value;
        }
    }
    ...
}

```

```

Buildings empireState = new Buildings (...);
empireState [102] = new Story ("O mais alto", ...);

```

Delegates

Um delegate pode ser pensado como um apontador para função orientado a objetos e type-safe (uma função $f: S \rightarrow R$ é dita ser type safe se a execução de f não gera um valor fora de R), que é capaz de guardar múltiplos métodos em vez de apenas um. Delegates tratam problemas que seriam resolvidos com apontadores de função em C++, e interfaces em Java. Ele melhora a abordagem de apontadores de função em C++ por ser type safe e por ser capaz de guardar múltiplos métodos.

Ele melhora a abordagem de interfaces por permitir a invocação de um método sem a necessidade de adaptadores de classes internas ou código extra para tratar invocações de métodos múltiplos. O uso mais importante de delegates é no tratamento de eventos, que é tratado na próxima seção (e que dá um exemplo de como delegates são usados).

Eventos

C# fornece suporte direto para eventos. Embora o tratamento de eventos tenha sido uma parte fundamental da programação desde seu início, um esforço surpreendentemente pequeno tem sido feito pela maior parte das linguagens para formalizar este conceito. Se você olhar como os maiores frameworks atuais tratam eventos, obteremos exemplos como os apontadores de função do Delphi (chamados closures), adaptadores de classes internas em Java e, é claro, o sistema de mensagens da API do Windows. C# usa delegates junto com a palavra-reservada event para fornecer uma solução bastante limpa para tratamento de eventos. A melhor forma de ilustrar isto é dando um exemplo que mostre todo o processo de declaração, disparo e tratamento do evento:

```

// A declaração do Delegate que define a assinatura
// dos métodos que podem ser invocados
public delegate void ScoreChangeEventHandler (int newScore, ref bool cancel);

// Classe que faz o evento
public class Game {
    // Repare no uso da palavra-reservada event
    public event ScoreChangeEventHandler ScoreChange;

    int score;

    // Propriedade Score
    public int Score {
        get {
            return score;
        }
        set {
            if (score != value) {
                bool cancel = false;
                ScoreChange (value, ref cancel);
                if (! cancel)
                    score = value;
            }
        }
    }
}

```

```

    }
    }
}

// Classe que trata o evento
public class Referee
{
    public Referee (Game game) {
        // Monitora quando um score é alterado no jogo
        game.ScoreChange += new ScoreChangeEventHandler (game_ScoreChange);
    }

    // Note como a assinatura deste método combina com a assinatura de
    ScoreChangeEventHandler
    private void game_ScoreChange (int newScore, ref bool cancel) {
        if (newScore < 100)
            System.Console.WriteLine ("Bom Score");
        else {
            cancel = true;
            System.Console.WriteLine ("Nenhum Score pode ser tão alto!");
        }
    }
}

// Classe para testar tudo
public class GameTest
{
    public static void Main () {
        Game game = new Game ();
        Referee referee = new Referee (game);
        game.Score = 70;
        game.Score = 110;
    }
}

```

Em GameTest criamos um game, criamos um referee (juiz) que monitora o jogo, e então alteramos o placar do jogo para ver o que o juiz faz em resposta a isto. Com este sistema, Game não tem conhecimento de Referee, e simplesmente permite que qualquer classe escute e aja sobre as mudanças feitas ao seu placar. A palavra-reservada event oculta todos os métodos de delegate adicionados com += e -= nas classes diferentes da que ele foi declarado. Esses operadores permitem que você adicione (e remova) quantos tratadores de evento você quiser ao evento.

Você provavelmente encontrará este sistema em frameworks de GUIs, onde o Game seria análogo a objetos de interface que disparam eventos de acordo com a entrada do usuário, e o Referee seria análogo a um form, que processaria os eventos.

Delegates foram introduzidos primeiramente no Visual J++ da Microsoft por Anders Hejlsberg, e foi um causa de uma grande disputa técnica e legal entre Sun e Microsoft.

Enums

No caso de você não conhecer C, Enuns permitem que você especifique um grupo de objetos, por exemplo:

Declaração

```
public enum Direction {North, East, West, South};
```


Uso:

```
Direction wall = Direction.North;
```

É uma construção muito boa, então a questão talvez não seja por que C# decidiu tê-la, mas em vez disso, por que Java preferiu omiti-la? Em Java, você teria

que ter:

Declaração:

```
public class Direction {  
    public final static int NORTH = 1;  
    public final static int EAST = 2;  
    public final static int WEST = 3;  
    public final static int SOUTH = 4;  
}
```

Uso:

```
int wall = Direction.NORTH;
```

Independente do fato da versão em Java parecer mais clara, ela não é, e é menos type-safe por permitir que você atribua acidentalmente a wall qualquer valor int sem que o compilador reclame. Um benefício de C# é boa surpresa que você tem ao depurar - se você colocar um breakpoint em uma enumeração que guarda enumerações combinadas, ela automaticamente decodificará a direção para dar a você uma saída humanamente legível, em vez de um número que você codificou:

Declaração:

```
public enum Direction {North=1, East=2, West=4, South=8};  
Usage:  
Direction direction = Direction.North | Direction.West;  
if ((direction & Direction.North) != 0)  
    ....
```

Se você colocar um breakpoint na instrução if, você verá uma versão legível para a direção em vez do número 5.

A melhor razão pela qual enums são ausentes em Java é que você pode obter resultado semelhante usando classes. Contudo, apenas com classes não somos capazes de expressar uma característica do mundo tão bem quanto poderíamos com uma outra construção. A filosofia do Java é "se isto pode ser feito por uma classe, então não introduza uma nova construção"?

Coleções e a Instrução Foreach

C# fornece uma alternativa para laços for, que também aumenta a consistência para classes de coleção: Em Java ou C++:

```
1. while (! collection.isEmpty()) {  
    Object o = collection.get();  
    collection.next();  
    ...  
2. for (int i = 0; i < array.length; i++)...
```

Em C#:

1. foreach (object o in collection)...
2. foreach (int i in array)...

O laço for do C# funcionará em objetos de coleção (arrays implementam uma coleção). Objetos de Coleção tem um método GetEnumerator() que retorna um objeto Enumerator. Um objeto Enumerator tem um método MoveNext e uma propriedade Current.

Structs

Os structs do C# são construções que fazem o sistema de tipos do C# funcionar elegantemente, e não apenas "uma forma de escrever código eficiente se você precisar".

Em C++, tanto structs quanto classes (objetos) podem ser alocados na pilha/in-line ou no heap. Em C#, structs são sempre criados na pilha/in-line, e classes (objetos) são sempre criados no heap. Structs permitem que um código mais eficiente seja criado:

```
public struct Vector {  
    public float direction;  
    public int magnitude;  
}
```

```
Vector[] vectors = new Vector [1000];
```

Isto alocará o espaço para 1000 vetores de uma vez, o que é muito mais eficiente do que se tivéssemos declarado Vector como uma classe e usado um for-loop para instanciar 1000 vetores individuais. O que fizemos aqui foi declarar um array como se declarássemos um array de int em C# ou Java:

```
int[] ints = new ints [1000];
```

C# simplesmente permite que você extenda o conjunto de tipos primitivos embutidos na linguagem. Na verdade, C# implementa todos os tipos primitivos como structs. O tipo int é um alias da struct System.Int32, o tipo long é um alias da struct System.Int64, etc. Esses tipos primitivos recebem, é claro, um tratamento especial por parte do compilador, mas a linguagem em si não faz tal distinção. A próxima seção mostra como C# tira vantagem disto.

Unificação de Tipos

Muitas linguagens têm tipos primitivos (int, long, etc), e tipos de alto nível que são compostos de tipos primitivos. Geralmente é útil ser capaz de tratar tipos primitivos e de alto nível da mesma forma. Por exemplo, é útil ter coleções que podem receber tanto ints quanto strings. Smalltalk obtém isso sacrificando alguma eficiência e tratando ints e longs como tipos como String ou Form. Java tenta evitar sacrificar esta eficiência, e trata tipos primitivos como no C ou C++, mas fornece classes de encapsulamento correspondentes para cada tipo primitivo - int é encapsulada por Integer, double é encapsulada por Double. Templates do C++ permitem which are ultimately composed of primitive types. It is often useful to be able to treat primitive types and higher level types in the same way. For instance, it is useful to have collections which can take both ints as well as strings. Smalltalk achieves this by sacrificing some efficiency and treating ints and longs as types like String or Form. Java tries to avoid sacrificing this efficiency, and treats primitive types like in C or C++, but provides corresponding wrapper classes for each primitive - int is wrapped by Integer, double is wrapped by Double. Templates em C++ permitem que seja escrito código que receba qualquer tipo, desde que as operações feitas sobre aquele tipo sejam realmente possíveis para ele. C# fornece uma solução diferente para este problema. Na seção anterior, mostramos os structs em C#, explicando como tipos primitivos como int

são apelidos para structs. Isto permite que código como este seja escrito, uma vez que structs possuem todos os métodos que a classe objeto possui:

```
int i = 5;  
System.Console.WriteLine (i.ToString());
```

Se você quiser usar uma struct como um objeto, C# 'encapsulará' a struct em um objeto para você, e restaurará a struct quando você precisar dela novamente:

```
Stack stack = new Stack ();  
stack.Push (i); // encapsula o int  
int j = (int) stack.Pop(); // restaura o int
```

Tirando o type-cast necessário quando restauramos as structs, esta é uma forma simples de tratar do relacionamento entre structs e classes. O criadores do C# devem ter considerado templates durante seu projeto. Deve ter havido duas razões principais para não usá-los. A primeira é a falta de organização - templates podem ser difíceis de juntar com características orientadas a objetos, eles abrem muitas possibilidades de projeto para os programadores, e difícil usá-los conscientemente. A segunda é que templates não seriam muito úteis a menos que as bibliotecas do .NET como classes de coleção os usassem. Contudo, se as classes do .NET os usassem, então as mais de 20 linguagens que usam as classes do .NET teriam que trabalhar com templates também, o que pode ser tecnicamente muito difícil de se conseguir.

É interessante notar que estão considerando templates (genéricos) para inclusão na linguagem Java pelo Java Community Process. Segundo uma entrevista com Anders Hejlsberg, parece que templates estão no horizonte, mas não para as primeiras versões, devido as dificuldades que foram discutidas acima. É interessante ver que a especificação da IL foi escrita de forma que o código da IL possa representar templates, enquanto que o byte-code não foi. Abaixo links para o Java Community Process considerando o uso de templates em Java e para a entrevista com Anders Hejlsberg:

- # Interview With Anders Hejlsberg and Tony Goodhew
- # Java Community Process for adding Generics to Java

Sobrecarga de Operador

Sobrecarga de operadores permite que os programadores construam tipos que sejam tão naturais de se usar quanto os tipos simples (int, long, etc.). C# implementa uma versão mais rígida da sobrecarga de operadores existente em C++, mas permite que classes tais como classes de números complexos, funcionem bem. Em C#, o operador == é um método não-virtual (operadores não podem ser virtuais) da classe do objeto que compara por referência. Quando você constrói uma classe, você pode definir seu próprio operador ==. Se você estiver usando sua classe com coleções, então você deve implementar a interface IComparable. Esta interface tem um método que deve ser implementado, chamado CompareTo (object), que deve retornar positivo, negativo, ou 0 se "this" é maior, menor ou de mesmo valor que o objeto. Você pode definir os métodos <, <=, >= e > se você quiser que os usuários da sua classe tenham uma sintaxe melhor. Os tipos numéricos (int, long, etc) implementam a interface IComparable.

A seguir, um exemplo simples de como tratar igualdades e comparações

```
public class Score : IComparable  
{  
    int value;
```

```
public Score (int score) {  
    value = score;  
}  
  
public static bool operator == (Score x, Score y) {  
    return x.value == y.value;  
}  
  
public static bool operator != (Score x, Score y) {  
    return x.value != y.value;  
}  
  
public int CompareTo (object o) {  
    return value - ((Score)o).value;  
}  
}  
  
Score a = new Score (5);  
Score b = new Score (5);  
Object c = a;  
Object d = b;
```

Para comparar a e b por referência:

```
System.Console.WriteLine ((object)a == (object)b; // false
```

Para comparar a e b por valor:

```
System.Console.WriteLine (a == b); // true
```

Para comparar c e d por referência:

```
System.Console.WriteLine (c == d); // false
```

Para comparar c e b por valor:

```
System.Console.WriteLine (((IComparable)c).CompareTo (d) == 0); // true
```

Você poderia também adicionar os operadores <, <=, >= e > à classe score.

C# garante que em tempo de compilação que operadores que são logicamente casados (!= e ==, > e <, >= and <=), devem ser ambos definidos.

Exercício 1:

B2-5 1

Hierarquia de Classes pode ser subdivida em dois grupos :

A)

generalização e agrupamento.

- B)
agrupamento e especialização.
- C)
derivação e especialização.
- D)
generalização e derivação.
- E)
generalização e especialização.

Exercício 2:

B2-6 1

Escolha a alternativa que representa corretamente a instância de novo um objeto em C#:

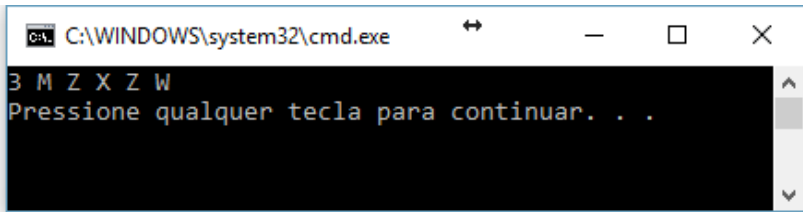
- A)
Dim a as new Objeto();
- B)
Dim a = new Objeto();
- C)
Objeto a as new Objeto();
- D)
Objeto a = new Objeto();
- E)
Objeto a = Object()

Exercício 3:

Dado o código abaixo, qual a saída?

```
1  public class Program
2  {
3      public static void Main(string[] args)
4      {
5          int[] v = new int[5];
6          for (int i = 0; i < 5; i++)
7          {
8              v[i] = i * 2;
9          }
10
11         try
12         {
13             TrataExemplo(v[4], 0);
14             TrataExemplo(v[6], 2);
15             TrataExemplo(v[3], 2);
16         }
17         catch (Exception)
18         {
19             Console.Write("W ");
20         }
21     }
22     public static void TrataExemplo(int i, int j)
23     {
24         try
25         {
26             Console.Write(i / j + " ");
27             Console.Write("M ");
28         }
29         catch (ArithmeticException)
30         {
31             Console.Write("X ");
32         }
33         catch (Exception)
34         {
35             Console.Write("Y ");
36         }
37         finally
38         {
39             Console.Write("Z ");
40         }
41     }
42 }
```

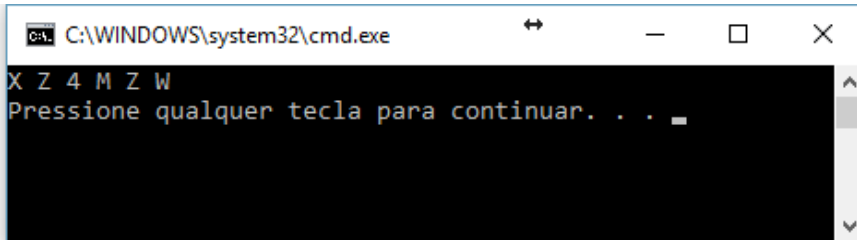
A)



```
C:\WINDOWS\system32\cmd.exe
3 M Z X Z W
Pressione qualquer tecla para continuar. . .
```

5, 5x[6-9], 11, 13, 22, 24, 26, 27, 37, 39, 14, 22, 24, 26, 29, 31, 37, 39, 15, 17, 19

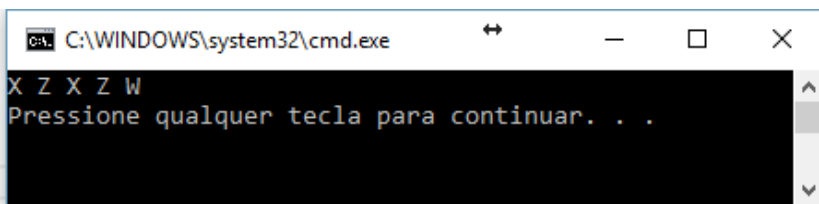
B)



```
C:\WINDOWS\system32\cmd.exe
X Z 4 M Z W
Pressione qualquer tecla para continuar. . .
```

5, 5x[6-9], 11, 13, 22, 24, 26, 29, 31, 37, 39, 14, 22, 24, 26, 27, 37, 39, 15, 17, 19

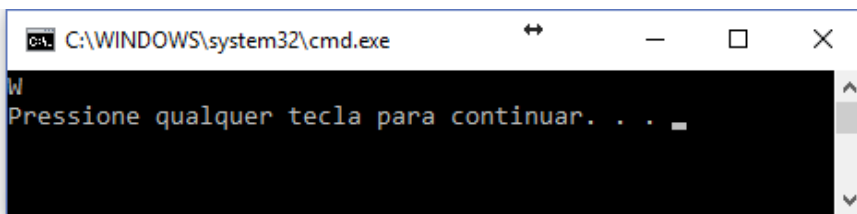
C)



```
C:\WINDOWS\system32\cmd.exe
X Z X Z W
Pressione qualquer tecla para continuar. . .
```

5, 5x[6-9], 11, 13, 22, 24, 26, 29, 31, 37, 39, 14, 22, 24, 26, 29, 31, 37, 39, 15, 17, 19

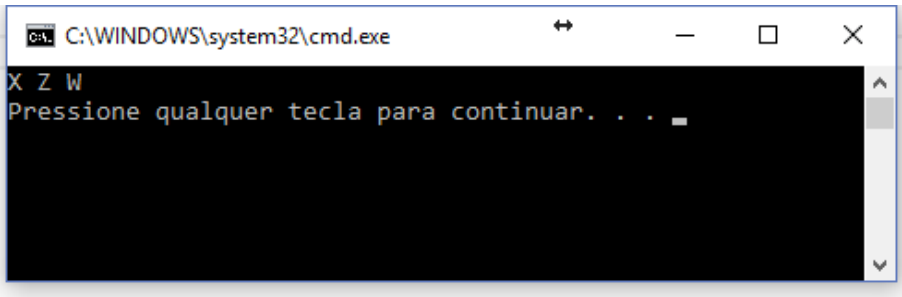
D)



```
C:\WINDOWS\system32\cmd.exe
W
Pressione qualquer tecla para continuar. . .
```

5, 5x[6-9], 11, 13, 17, 19

E)



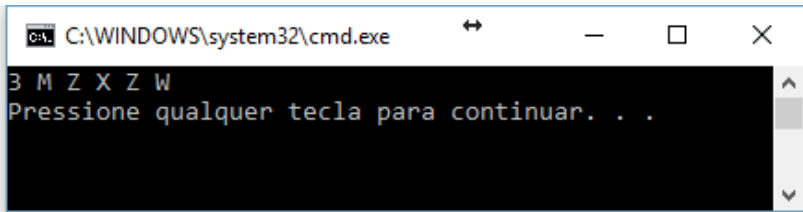
5, 5x[6-9], 11, 13, 22, 24, 26, 29, 31, 37, 39, 14, 17, 19

Exercício 4:

Considerando o código abaixo, qual a saída ao executá-lo?

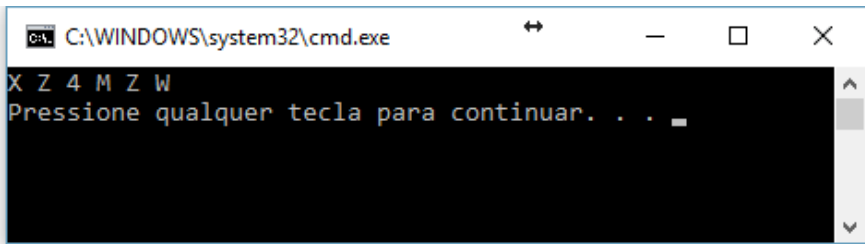

```
1      public class Program
2      {
3          public static void Main(string[] args)
4          {
5              int[] v = new int[5];
6              for (int i = 0; i < 5; i++)
7              {
8                  v[i] = i * 2;
9              }
10
11              try
12              {
13                  TrataExemplo(v[3], 2);
14                  TrataExemplo(v[4], 0);
15                  TrataExemplo(v[6], 2);
16              }
17              catch (Exception)
18              {
19                  Console.Write("W ");
20              }
21          }
22          public static void TrataExemplo(int i, int j)
23          {
24              try
25              {
26                  Console.Write(i / j + " ");
27                  Console.Write("M ");
28              }
29              catch (ArithmeticException)
30              {
31                  Console.Write("X ");
32              }
33              catch (Exception)
34              {
35                  Console.Write("Y ");
36              }
37              finally
38              {
39                  Console.Write("Z ");
40              }
41          }
42      }
```

A)



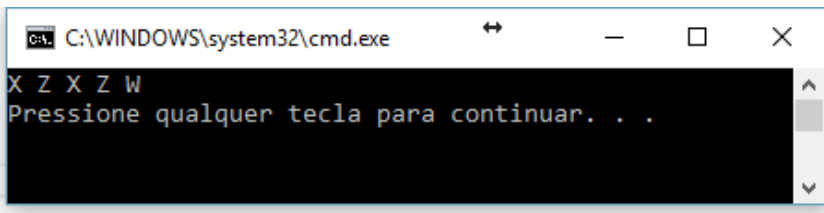
```
C:\WINDOWS\system32\cmd.exe
3 M Z X Z W
Pressione qualquer tecla para continuar. . .
```

B)



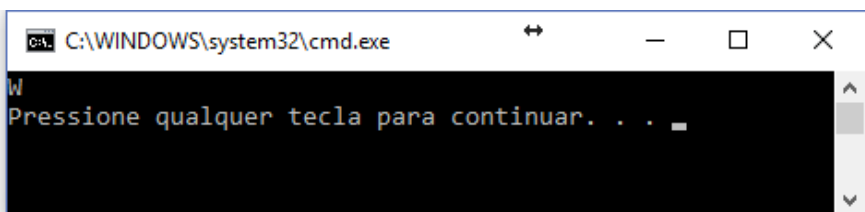
```
C:\WINDOWS\system32\cmd.exe
X Z 4 M Z W
Pressione qualquer tecla para continuar. . .
```

C)



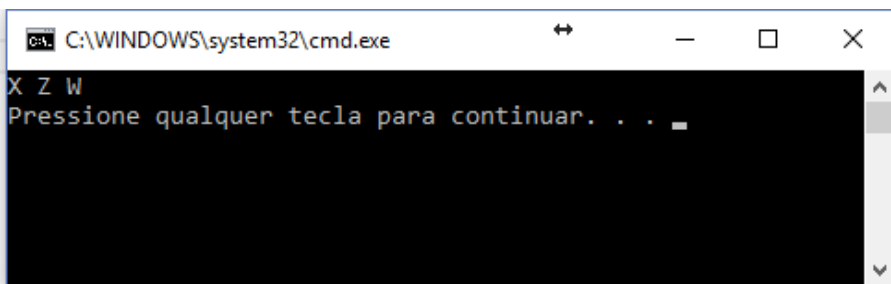
```
C:\WINDOWS\system32\cmd.exe
X Z X Z W
Pressione qualquer tecla para continuar. . .
```

D)



```
C:\WINDOWS\system32\cmd.exe
W
Pressione qualquer tecla para continuar. . .
```

E)



```
C:\WINDOWS\system32\cmd.exe
X Z W
Pressione qualquer tecla para continuar. . .
```

Exercício 5:

Qual dos métodos abaixo não faz parte da classe Object?

A)

Equals

B)

Finally

C)

Finalise

D)

GetHashCode

E)

MemberWiseClone

Exercício 6:

Qual das afirmações NÃO caracteriza a classe Object

A)

suporta todas as classes na hierarquia de classes

B)

Alguns dos seus métodos são ToString, Try, Catch, Finally, GetType

C)

é classe base fundamental de todas as classes

D)

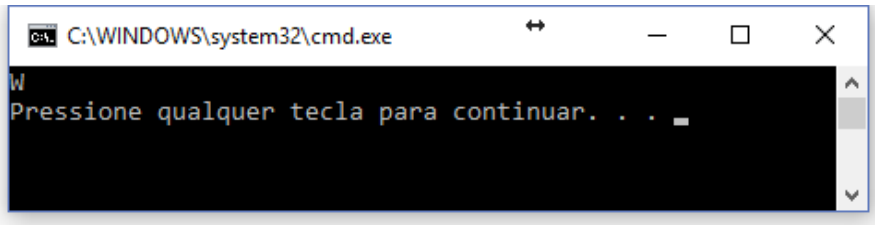
todos os métodos dele estão disponíveis em todos os demais objetos no sistema

E)

fornece serviços de baixo nível para classes derivadas

Exercício 7:

Considerando a saída abaixo, qual o código que o gerou?



A)

```
1      public class Program
2      {
3          public static void Main(string[] args)
4          {
5              int[] v = new int[5];
6              for (int i = 0; i < 5; i++)
7              {
8                  v[i] = i * 2;
9              }
10
11              try
12              {
13                  TrataExemplo(v[3], 2);
14                  TrataExemplo(v[4], 0);
15                  TrataExemplo(v[6], 2);
16              }
17              catch (Exception)
18              {
19                  Console.Write("W ");
20              }
21          }
22          public static void TrataExemplo(int i, int j)
23          {
24              try
25              {
26                  Console.Write(i / j + " ");
27                  Console.Write("M ");
28              }
29              catch (ArithmeticException)
30              {
31                  Console.Write("X ");
32              }
33              catch (Exception)
34              {
35                  Console.Write("Y ");
36              }
37              finally
38              {
39                  Console.Write("Z ");
40              }
41          }
42      }
```

B)

```
1      public class Program
2      {
3          public static void Main(string[] args)
4          {
5              int[] v = new int[5];
6              for (int i = 0; i < 5; i++)
7              {
8                  v[i] = i * 2;
9              }
10
11             try
12             {
13                 TrataExemplo(v[3], 0);
14                 TrataExemplo(v[4], 2);
15                 TrataExemplo(v[6], 2);
16             }
17             catch (Exception)
18             {
19                 Console.Write("W ");
20             }
21         }
22         public static void TrataExemplo(int i, int j)
23         {
24             try
25             {
26                 Console.Write(i / j + " ");
27                 Console.Write("M ");
28             }
29             catch (ArithmeticException)
30             {
31                 Console.Write("X ");
32             }
33             catch (Exception)
34             {
35                 Console.Write("Y ");
36             }
37             finally
38             {
39                 Console.Write("Z ");
40             }
41         }
42     }
```

C)

```
1      public class Program
2      {
3          public static void Main(string[] args)
4          {
5              int[] v = new int[5];
6              for (int i = 0; i < 5; i++)
7              {
8                  v[i] = i * 2;
9              }
10
11             try
12             {
13                 TrataExemplo(v[3], 0);
14                 TrataExemplo(v[4], 0);
15                 TrataExemplo(v[6], 2);
16             }
17             catch (Exception)
18             {
19                 Console.Write("W ");
20             }
21         }
22         public static void TrataExemplo(int i, int j)
23         {
24             try
25             {
26                 Console.Write(i / j + " ");
27                 Console.Write("M ");
28             }
29             catch (ArithmeticException)
30             {
31                 Console.Write("X ");
32             }
33             catch (Exception)
34             {
35                 Console.Write("Y ");
36             }
37             finally
38             {
39                 Console.Write("Z ");
40             }
41         }
42     }
```

D)

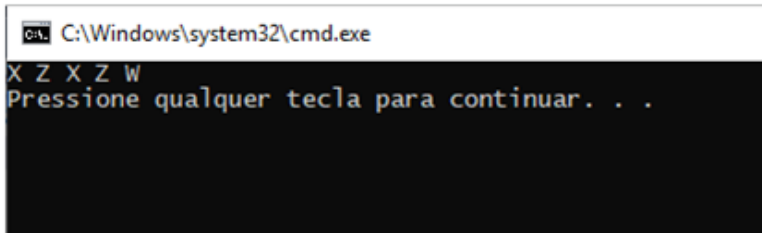
```
1      public class Program
2      {
3          public static void Main(string[] args)
4          {
5              int[] v = new int[5];
6              for (int i = 0; i < 5; i++)
7              {
8                  v[i] = i * 2;
9              }
10
11              try
12              {
13                  TrataExemplo(v[6], 2);
14                  TrataExemplo(v[4], 0);
15                  TrataExemplo(v[3], 2);
16              }
17              catch (Exception)
18              {
19                  Console.Write("W ");
20              }
21          }
22          public static void TrataExemplo(int i, int j)
23          {
24              try
25              {
26                  Console.Write(i / j + " ");
27                  Console.Write("M ");
28              }
29              catch (ArithmeticException)
30              {
31                  Console.Write("X ");
32              }
33              catch (Exception)
34              {
35                  Console.Write("Y ");
36              }
37              finally
38              {
39                  Console.Write("Z ");
40              }
41          }
42      }
```


E)

```
1      public class Program
2      {
3          public static void Main(string[] args)
4          {
5              int[] v = new int[5];
6              for (int i = 0; i < 5; i++)
7              {
8                  v[i] = i * 2;
9              }
10
11             try
12             {
13                 TrataExemplo(v[4], 0);
14                 TrataExemplo(v[6], 2);
15                 TrataExemplo(v[3], 2);
16             }
17             catch (Exception)
18             {
19                 Console.Write("W ");
20             }
21         }
22         public static void TrataExemplo(int i, int j)
23         {
24             try
25             {
26                 Console.Write(i / j + " ");
27                 Console.Write("M ");
28             }
29             catch (ArithmeticException)
30             {
31                 Console.Write("X ");
32             }
33             catch (Exception)
34             {
35                 Console.Write("Y ");
36             }
37             finally
38             {
39                 Console.Write("Z ");
40             }
41         }
42     }
```

Exercício 8:

Considere a saída abaixo, qual o código que o gerou?



A)

```
1      public class Program
2      {
3          public static void Main(string[] args)
4          {
5              int[] v = new int[5];
6              for (int i = 0; i < 5; i++)
7              {
8                  v[i] = i * 2;
9              }
10
11              try
12              {
13                  TrataExemplo(v[3], 2);
14                  TrataExemplo(v[4], 0);
15                  TrataExemplo(v[6], 2);
16              }
17              catch (Exception)
18              {
19                  Console.Write("W ");
20              }
21          }
22          public static void TrataExemplo(int i, int j)
23          {
24              try
25              {
26                  Console.Write(i / j + " ");
27                  Console.Write("M ");
28              }
29              catch (ArithmeticException)
30              {
31                  Console.Write("X ");
32              }
33              catch (Exception)
34              {
35                  Console.Write("Y ");
36              }
37              finally
38              {
39                  Console.Write("Z ");
40              }
41          }
42      }
```

B)

```
try
{
    // Código sujeito a exceções
}
catch (TipoExcecao1 e)
{
    // Tratamento para exceção tipo 1
}
catch (TipoExcecao2 e)
{
    // Tratamento para exceção tipo 2
}
catch
{
    // Tratamento para qualquer tipo de exceção
}
finally
{
    // Trecho que deve sempre ser executado, havendo ou não exceção
}
```

C)

```
1      public class Program
2      {
3          public static void Main(string[] args)
4          {
5              int[] v = new int[5];
6              for (int i = 0; i < 5; i++)
7              {
8                  v[i] = i * 2;
9              }
10
11              try
12              {
13                  TrataExemplo(v[3], 0);
14                  TrataExemplo(v[4], 0);
15                  TrataExemplo(v[6], 2);
16              }
17              catch (Exception)
18              {
19                  Console.Write("W ");
20              }
21          }
22          public static void TrataExemplo(int i, int j)
23          {
24              try
25              {
26                  Console.Write(i / j + " ");
27                  Console.Write("M ");
28              }
29              catch (ArithmeticException)
30              {
31                  Console.Write("X ");
32              }
33              catch (Exception)
34              {
35                  Console.Write("Y ");
36              }
37              finally
38              {
39                  Console.Write("Z ");
40              }
41          }
42      }
```

D)

```
1      public class Program
2      {
3          public static void Main(string[] args)
4          {
5              int[] v = new int[5];
6              for (int i = 0; i < 5; i++)
7              {
8                  v[i] = i * 2;
9              }
10
11              try
12              {
13                  TrataExemplo(v[6], 2);
14                  TrataExemplo(v[4], 0);
15                  TrataExemplo(v[3], 2);
16              }
17              catch (Exception)
18              {
19                  Console.Write("W ");
20              }
21          }
22          public static void TrataExemplo(int i, int j)
23          {
24              try
25              {
26                  Console.Write(i / j + " ");
27                  Console.Write("M ");
28              }
29              catch (ArithmeticException)
30              {
31                  Console.Write("X ");
32              }
33              catch (Exception)
34              {
35                  Console.Write("Y ");
36              }
37              finally
38              {
39                  Console.Write("Z ");
40              }
41          }
42      }
```

E)

```
1  public class Program
2  {
3      public static void Main(string[] args)
4      {
5          int[] v = new int[5];
6          for (int i = 0; i < 5; i++)
7          {
8              v[i] = i * 2;
9          }
10
11         try
12         {
13             TrataExemplo(v[4], 0);
14             TrataExemplo(v[6], 2);
15             TrataExemplo(v[3], 2);
16         }
17         catch (Exception)
18         {
19             Console.Write("W ");
20         }
21     }
22     public static void TrataExemplo(int i, int j)
23     {
24         try
25         {
26             Console.Write(i / j + " ");
27             Console.Write("M ");
28         }
29         catch (ArithmeticException)
30         {
31             Console.Write("X ");
32         }
33         catch (Exception)
34         {
35             Console.Write("Y ");
36         }
37         finally
38         {
39             Console.Write("Z ");
40         }
41     }
42 }
```